Rekursion – Konzepte, Strukturen und Umsetzung in funktionalen Programmiersprachen

aus dem Studiengang Wirtschaftsinformatik

an der Duale Hochschule Baden-Württemberg Mannheim

von

Emanuel Schuessler

Jannes Borgmann

08.05.2026

Matrikelnummer, Kurs: ???, WI25SEA
???, WI25SEA

Unternehmen: SAP SE, 69190, Walldorf
SAP SE, 69190, Walldorf

Betreuer an der DHBW: Edelbert Schumm

Sperrvermerk

Die vorliegende Arbeit mit dem Titel

Rekursion – Konzepte, Strukturen und Umsetzung in funktionalen Programmiersprachen

enthält unternehmensinterne bzw. vertrauliche Informationen der SAP SE, ist deshalb mit einem Sperrvermerk versehen und wird ausschließlich zu Prüfungszwecken im Studiengang Wirtschaftsinformatik der Duale Hochschule Baden-Württemberg Mannheim vorgelegt.

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte (SAP SE) vorliegt.

Walldorf 08.05.2026

Emanuel Schuessler

Jannes Borgmann

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Wir versichern hiermit, dass wir unsere Arbeit mit dem Thema:

Rekursion – Konzepte, Strukturen und Umsetzung in funktionalen Programmiersprachen

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Wir versichern zudem, dass alle eingereichten Fassungen übereinstimmen.

Walldorf 08.05.2026

Emanuel Schuessler

Jannes Borgmann

Inhaltsverzeichnis

1 Einleitung **1**

2 Definition und Grundlagen der Rekursion **2**
2.1 Was ist Rekursion? 2
2.2 Historischer Hintergrund 2
2.3 Strukturelle Rekursion und induktive Datentypen 3

3 Rekursive Funktionen **5**
3.1 Grundprinzip und das Fakultätsbeispiel 5
3.2 Terminierung und Korrektheit 6
3.3 Endrekursion 6
3.4 Gegenseitige und verschachtelte Rekursion 7

4 Rekursive Datenstrukturen **8**
4.1 Das Konzept induktiver Datentypen 8
4.2 Listen 8
4.3 Bäume 9

5 Rekursion im Kontext der funktionalen Programmierung **10**
5.1 Rekursion als Ersatz für Schleifen 10
5.2 Lazy Evaluation und unendliche Strukturen 10
5.3 Verbindung zum Lambda-Kalkül 10
5.4 Höherwertige Funktionen als verallgemeinerte Rekursion 11

6 Fazit **12**

Literatur ..... **a**

Tabellenverzeichnis

Tabelle 1 Strukturelle Parallele zwischen induktivem Datentyp und rekursiver Funktion **4**

Codeverzeichnis

Listing 1 Rekursive Fakultät in Java **5**

Abkürzungsverzeichnis

BNF	Backus-Naur-Form
FP	Funktionale Programmierung
GGT	Größter Gemeinsamer Teiler

Glossar

Basisfall Der Abbruchfall einer rekursiven Funktion, der ohne weiteren Selbstaufruf einen Wert zurückgibt und so die Terminierung der Rekursion sicherstellt.

Endrekursion Eine spezielle Form der Rekursion, bei der der rekursive Aufruf die letzte Operation der Funktion ist. Viele Compiler können Endrekursion in eine Schleife umwandeln (Tail Call Optimization).

Pattern_Matching Ein Sprachkonstrukt in funktionalen Programmiersprachen, das es erlaubt, Werte anhand ihrer Struktur zu unterscheiden und zu destrukturieren.

Rekursion Ein Konzept, bei dem sich eine Funktion oder Datenstruktur direkt oder indirekt auf sich selbst bezieht. Eine rekursive Definition besteht stets aus einem Basisfall (Abbruchbedingung) und einem allgemeinen Fall (Selbstaufruf).

Selbstaufruf Der Teil einer rekursiven Funktion, in dem die Funktion sich selbst mit einem vereinfachten Argument aufruft.

Strukturelle_Rekursion Eine Form der Rekursion, die der Struktur des Eingabedatentyps folgt, z.B. bei der Verarbeitung von Listen oder Bäumen.

1 Einleitung

Rekursion ist eines der grundlegendsten und gleichzeitig faszinierendsten Konzepte der Informatik und Mathematik. Sie beschreibt ein Prinzip, bei dem etwas nicht explizit, sondern durch einfachere Versionen seiner selbst definiert wird [1]. Was auf den ersten Blick wie ein logischer Zirkel wirkt, erweist sich bei näherer Betrachtung als ein äußerst mächtiges Werkzeug: Viele Probleme, die sich iterativ nur umständlich beschreiben lassen, besitzen eine natürliche, elegante rekursive Formulierung.

Im Alltag begegnet uns Rekursion häufiger, als wir es zunächst vermuten würden. Das Blätterwerk eines Baumes, die Verzweigungen eines Flusssystems oder Schneeflockenmuster folgen rekursiven Bildungsgesetzen. In der Mathematik tritt Rekursion schon in der Schule auf, etwa bei der Definition von Folgen oder bei der Fakultätsfunktion – auch wenn der Begriff selbst dort oft vermieden wird. In der Informatik und insbesondere in der funktionalen Programmierung nimmt Rekursion eine zentrale Rolle ein: Sie ist das primäre Mittel zur Beschreibung von Wiederholungen, da rein funktionale Sprachen wie Haskell auf veränderliche Zustandsvariablen und klassische Schleifen verzichten [2].

Diese Ausarbeitung beleuchtet zunächst die theoretischen Grundlagen und die Geschichte der Rekursion, erläutert anschließend rekursive Funktionen und rekursive Datenstrukturen und zeigt schließlich deren konkrete Umsetzung in der funktionalen Programmiersprache Haskell. Abschließend werden Querverbindungen zu verwandten Themen wie Listen, Datenstrukturen und dem Lambda-Kalkül hergestellt.

2 Definition und Grundlagen der Rekursion

2.1 Was ist Rekursion?

Rekursion bezeichnet in der Mathematik und Informatik ein Konstruktionsprinzip, bei dem eine Definition unmittelbar oder mittelbar auf sich selbst verweist. Eine oft zitierte, informelle Beschreibung lautet: *„Man definiert etwas nicht explizit, sondern durch einfachere Versionen seiner selbst„* [1]. Entscheidend dabei ist das Wort „einfacher“: Damit eine rekursive Definition sinnvoll ist und nicht ins Unendliche führt, muss jeder Selbstaufruf das Problem auf eine kleinere, einfachere Variante zurückführen.

Jede korrekte rekursive Definition besteht daher aus mindestens zwei Bestandteilen. Der erste ist der **Basisfall**, auch Verankerung oder Abbruchbedingung genannt. Er beschreibt einen Spezialfall, der sich direkt und ohne weiteren Selbstaufruf lösen lässt. Er stellt sicher, dass die Rekursion irgendwann terminiert. Der zweite Bestandteil ist der **allgemeine Fall**, auch Vererbung genannt. Hier wird das eigentliche Problem auf eine einfachere Version desselben Problems zurückgeführt, indem die Funktion sich selbst mit einem reduzierten Argument aufruft – dem sogenannten Selbstaufruf. Das allgemeine Standard-Rekursionsschema lässt sich formal wie folgt darstellen [1]:

$$f(x) = \begin{cases} c(x) & \text{falls } P(x) \text{ (Basisfall)} \\ h(x, f(\varphi(x))) & \text{sonst} \quad \text{(allgemeiner Fall)} \end{cases} \tag{1}$$

Dabei bezeichnet $P(x)$ die Abbruchbedingung, φ die Vorgängerfunktion, welche das Argument in Richtung Basisfall verkleinert, und h die Schrittfunktion, die aus dem aktuellen Argument und dem Ergebnis des rekursiven Aufrufs den neuen Wert berechnet.

2.2 Historischer Hintergrund

Die Geschichte der Rekursion reicht weit zurück. Den ersten bekannten rekursiven Algorithmus lieferte Euklid im dritten Jahrhundert vor Christus mit dem Verfahren der wechselnden Wegnahme zur Berechnung des Größter Gemeinsamer Teiler (GGT) zweier ganzer Zahlen. Die Grundidee ist bestechend einfach: Haben zwei Stäbe der Längen a und b ein gemeinsames

Maß, so hat auch der kürzere Stab und die Differenz der beiden Stäbe ein gemeinsames Maß. Das Verfahren wird solange wiederholt, bis beide Stäbe gleich lang sind – das ist dann der gesuchte größte gemeinsame Teiler [1].

Im 19. Jahrhundert formalisierte Richard Dedekind den Begriff der rekursiven Definition im Rahmen der Peano-Axiome für die natürlichen Zahlen. Die Grundoperationen Addition und Multiplikation werden dort nicht durch explizite Formeln, sondern durch rekursive Schemata definiert. Im 20. Jahrhundert wurde Rekursion durch die Arbeiten von David Hilbert, Kurt Gödel und Alonzo Church zum zentralen Gegenstand der mathematischen Logik und der Berechenbarkeitstheorie. Der Begriff „rekursiv„ im modernen Sinne wurde erstmals von Gödel in seinen Arbeiten über primitive und allgemein rekursive Funktionen verwendet.

In der Informatik etablierte sich Rekursion spätestens mit der Entwicklung der Programmiersprache LISP in den späten 1950er Jahren als fundamentales Konzept. John McCarthy, der Entwickler von LISP, erkannte, dass sich viele Berechnungen auf natürliche Weise als rekursive Funktionsdefinitionen ausdrücken lassen – eine Einsicht, die die gesamte Entwicklung der funktionalen Programmierung prägte.

2.3 Strukturelle Rekursion und induktive Datentypen

Ein besonders wichtiges Konzept ist die **Strukturelle_Rekursion**. Sie tritt auf, wenn eine rekursive Funktion der Struktur ihres Eingabedatentyps folgt. Induktiv definierte Datentypen – also Typen, die entweder ein Basisobjekt oder eine Kombination aus einfacheren Objekten desselben Typs sind – und strukturell rekursive Funktionen über ihnen besitzen eine tiefe strukturelle Gleichheit [1]. Die natürlichen Zahlen etwa sind entweder die Null oder der Nachfolger einer natürlichen Zahl; eine Funktion, die über natürliche Zahlen rekursiv arbeitet, behandelt entsprechend den Fall Null als Basisfall und den Nachfolger-Fall als allgemeinen Fall. Diese Parallelität ist kein Zufall, sondern ein fundamentales Prinzip: Die Struktur korrekter rekursiver Programme spiegelt stets die Struktur der Daten wider, auf denen sie operieren.

Natürliche Zahlen (induktiv)	**Rekursive Funktion f(n)**
$0 \longrightarrow$ Basisfall	$f(0) = c_0 \longrightarrow$ Basisfall
Nachfolger $n' = n + 1$	$f(n) = h(n, f(n-1)) \longrightarrow$ allg. Fall

Tabelle 1 — Strukturelle Parallele zwischen induktivem Datentyp und rekursiver Funktion

3 Rekursive Funktionen

3.1 Grundprinzip und das Fakultätsbeispiel

Das bekannteste Einführungsbeispiel für eine rekursive Funktion ist die Fakultät. Mathematisch ist sie definiert als das Produkt aller positiven ganzen Zahlen bis einschließlich n. Statt diese Summe mit Pünktchenschreibweise anzudeuten, lässt sich die Fakultät präzise rekursiv erfassen: Die Fakultät von Null ist Eins (Basisfall), und die Fakultät einer beliebigen positiven Zahl n ist n multipliziert mit der Fakultät von $n-1$ (allgemeiner Fall). Diese Definition beschreibt exakt das Berechnungsverfahren: Um $n!$ zu berechnen, muss man erst $(n-1)!$ kennen, um $(n-1)!$ zu berechnen, muss man $(n-2)!$ kennen, und so fort, bis man beim Basisfall $0! = 1$ angelangt ist.

In Java lässt sich diese mathematische Definition direkt als Methode umschreiben. Die `if`-Verzweigung entspricht dabei exakt der mathematischen Fallunterscheidung: Der Basisfall $n = 0$ wird explizit abgefangen und liefert sofort den Wert 1 zurück. Für alle anderen Werte ruft sich die Methode mit dem um Eins verkleinerten Argument selbst auf und multipliziert das Ergebnis anschließend mit n.

```java
public static long factorial(int n) {
    if (n == 0) {
        return 1;            // Basisfall
    }
    return n * factorial(n - 1);  // allgemeiner Fall
}
```

Listing 1 — Rekursive Fakultät in Java

Der Aufruf `factorial(4)` löst dabei die folgende Aufrufkette aus: `factorial(4)` wartet auf `factorial(3)`, dieses auf `factorial(2)`, dieses auf `factorial(1)`, und erst `factorial(0)` liefert direkt 1 zurück. Danach werden die Ergebnisse in umgekehrter Reihenfolge zusammengesetzt: $1 \cdot 1 = 1$, $2 \cdot 1 = 2$, $3 \cdot 2 = 6$, $4 \cdot 6 = 24$.

3.2 Terminierung und Korrektheit

Eine zentrale Frage bei jeder rekursiven Funktion ist die der Terminierung: Kommt die Berechnung irgendwann zu einem Ende, oder läuft sie unendlich lange? Eine rekursive Funktion terminiert genau dann für alle Eingaben, wenn die Vorgängerfunktion φ das Argument monoton auf den Basisfall zuführt, also bei jedem Schritt eine wohldefinierte Größe strikt abnimmt [1]. Beim Fakultätsbeispiel ist diese Größe das Argument selbst: es wird bei jedem Aufruf um genau Eins verkleinert und erreicht schließlich den Basisfall Null.

Nicht jede rekursive Funktion terminiert für alle Eingaben. Sogenannte partiell berechenbare Funktionen sind nur für einen Teil ihrer möglichen Argumente definiert. Dies führt in der Praxis zum gefürchteten „Hängen„ eines Programms – das Programm antwortet nicht mehr, weil es in einem unendlichen Rekursionszyklus gefangen ist. Lorenz beschreibt dieses Phänomen treffend: *„Keine Antwort ist auch keine Antwort“* [1].

3.3 Endrekursion

Ein praktisches Problem naiver Rekursion ist der Speicherverbrauch. Bei jedem Funktionsaufruf legt das Laufzeitsystem einen neuen Eintrag auf dem Aufrufstapel (Call Stack) an, der den lokalen Zustand der Berechnung festhält. Bei tiefer Rekursion kann dieser Stapel überlaufen – ein sogenannter Stack Overflow. Die **Endrekursion** bietet eine elegante Lösung: Eine Funktion ist endrekursiv, wenn der rekursive Aufruf die allerletzte Operation der Funktion ist und dessen Ergebnis direkt zurückgegeben wird, ohne danach noch mit einem weiteren Wert verknüpft zu werden. In diesem Fall benötigt der Compiler den bisherigen Stackframe nicht mehr und kann ihn wiederverwenden, anstatt einen neuen anzulegen. Diese Optimierung heißt *Tail Call Optimization* und ist in Haskell sowie den meisten anderen funktionalen Sprachen garantiert. Endrekursive Funktionen sind damit in ihrem Speicherverbrauch äquivalent zu imperativen Schleifen.

Um die Fakultätsfunktion endrekursiv zu machen, wird ein Akkumulator eingeführt, der das bisherige Zwischenergebnis mitführt. Statt das Ergebnis des rekursiven Aufrufs noch mit n multiplizieren zu müssen, wird die Multiplikation bereits vor dem Aufruf durchgeführt und dem Akkumulator übergeben.

3.4 Gegenseitige und verschachtelte Rekursion

Neben der direkten Rekursion, bei der eine Funktion sich selbst aufruft, gibt es weitere Varianten. Bei der **gegenseitigen Rekursion** rufen sich zwei oder mehr Funktionen wechselseitig auf. Ein klassisches Beispiel sind die Funktionen `isEven` und `isOdd`: eine Zahl ist gerade, wenn ihr Vorgänger ungerade ist, und ungerade, wenn ihr Vorgänger gerade ist. Bei der **verschachtelten Rekursion** tritt der Selbstaufruf als Argument eines weiteren Selbstaufrufs auf. Das bekannteste Beispiel ist die Ackermann-Funktion, die zwar für alle natürlichen Zahlen terminiert, aber so schnell wächst, dass sie praktisch nicht berechenbar ist und auch nicht durch das primitive Rekursionsschema erfasst werden kann [1].

4 Rekursive Datenstrukturen

4.1 Das Konzept induktiver Datentypen

Nicht nur Funktionen, sondern auch Datenstrukturen können rekursiv definiert sein. Ein rekursiver Datentyp ist ein Typ, dessen Definition sich auf sich selbst bezieht – analog zur rekursiven Funktionsdefinition mit einem Basisfall und einem allgemeinen Fall. Solche Typen werden in der Informatik auch als **induktive Datentypen** bezeichnet, weil man alle möglichen Werte des Typs konstruktiv „aufzählen„ kann: Man beginnt mit dem Basisfall und erzeugt durch wiederholte Anwendung des allgemeinen Falls alle weiteren Werte [3].

Der Vorteil rekursiver Datentypen liegt in ihrer natürlichen Beschreibungskraft: Viele Probleme besitzen eine inhärent rekursive Struktur, und induktive Datentypen ermöglichen es, diese Struktur direkt im Programmcode abzubilden. Funktionale Sprachen wie Haskell haben diese Idee zur zentralen Grundlage ihres Typsystems gemacht.

4.2 Listen

Die Liste ist der prototypische rekursive Datentyp in der funktionalen Programmierung. Eine Liste ist entweder leer oder sie besteht aus einem ersten Element (dem Kopf) und einer weiteren Liste (dem Schwanz). Diese Definition ist vollständig induktiv: Der Basisfall ist die leere Liste, und der allgemeine Fall kombiniert ein beliebiges Element mit einer bereits bestehenden Liste zu einer neuen, längeren Liste [3].

In Haskell wird diese Struktur direkt durch Pattern Matching ausgenutzt. Funktionen über Listen folgen zwingend diesem Aufbau: Sie behandeln die leere Liste als Basisfall und zerlegen im allgemeinen Fall die Liste in Kopf und Schwanz. Die Länge einer Liste ist Null, wenn sie leer ist, andernfalls Eins plus die Länge des Schwanzes. Die Summe einer leeren Liste ist Null, andernfalls das erste Element plus die Summe der restlichen Elemente. Dieses Muster zieht sich durch praktisch alle Listenoperationen und macht den Code nicht nur knapp, sondern auch leicht nachvollziehbar.

4.3 Bäume

Bäume sind ein weiterer wichtiger rekursiver Datentyp, der in der Informatik allgegenwärtig ist – von Dateisystemen über Syntaxbäume von Programmiersprachen bis hin zu Datenbanken. Ein Binärbaum ist entweder ein leeres Blatt (Basisfall) oder ein Knoten, der einen Wert sowie einen linken und einen rechten Teilbaum enthält (allgemeiner Fall). Auch hier folgt die Struktur des Datentyps direkt der Struktur der rekursiven Funktionen, die ihn verarbeiten: Um die Tiefe eines Baumes zu bestimmen, berechnet man rekursiv die Tiefen des linken und rechten Teilbaums und nimmt das Maximum. Der Basisfall – ein leeres Blatt – hat die Tiefe Null.

5 Rekursion im Kontext der funktionalen Programmierung

5.1 Rekursion als Ersatz für Schleifen

In imperativen Programmiersprachen wie Java oder C werden Wiederholungen durch Schleifen ausgedrückt. Eine `for`-Schleife etwa wiederholt einen Block von Anweisungen, während eine Zählvariable verändert wird. In rein funktionalen Sprachen hingegen gibt es keine veränderbaren Variablen – ein Wert, dem einmal ein Ausdruck zugewiesen wurde, bleibt unveränderlich. Damit ist das Konzept einer Schleife mit veränderbarem Zustand grundsätzlich nicht realisierbar [2].

Rekursion übernimmt diese Rolle vollständig. Was imperative Programmierung durch Zustandsveränderung ausdrückt, drückt funktionale Programmierung durch Parameterübergabe aus: Statt eine Variable zu inkrementieren, übergibt man den neuen Wert als Argument an den nächsten rekursiven Aufruf. Das Ergebnis ist semantisch äquivalent, aber ohne jeden veränderbaren Zustand. Durch *Tail Call Optimization* ist diese Form der Rekursion auch hinsichtlich Laufzeit und Speicherbedarf mit Schleifen gleichwertig.

5.2 Lazy Evaluation und unendliche Strukturen

Ein bemerkenswertes Merkmal von Haskell ist die sogenannte *lazy evaluation* (bedarfsgesteuerte Auswertung): Ausdrücke werden erst dann berechnet, wenn ihr Wert tatsächlich benötigt wird. In Verbindung mit Rekursion ermöglicht dies die Definition potenziell unendlicher Datenstrukturen. Eine unendliche Liste aller natürlichen Zahlen lässt sich in Haskell direkt hinschreiben, da die Sprache nur so viele Elemente tatsächlich berechnet, wie später abgefragt werden. Dieses Konzept wird auch als *Stream* bezeichnet und verbindet Rekursion eng mit dem Thema Listen und Streams [4].

5.3 Verbindung zum Lambda-Kalkül

Im reinen Lambda-Kalkül, dem theoretischen Fundament der funktionalen Programmierung, gibt es zunächst keine direkte Möglichkeit, benannte, sich selbst aufrufende Funktionen zu definieren. Rekursion wird dort durch den **Y-Kombinator** realisiert, einen sogenannten

Fixpunktoperator. Er nimmt eine Funktion entgegen und gibt deren Fixpunkt zurück – also denjenigen Wert $f(x) = x$, der unter wiederholter Anwendung der Funktion unverändert bleibt. Mithilfe des Y-Kombinators lässt sich jede rekursive Funktion in eine äquivalente, nicht-rekursive Form im Lambda-Kalkül überführen. Dies beweist, dass Rekursion keine primitive Sprachfunktion sein muss, sondern sich aus einfachsten Grundbausteinen konstruieren lässt [2].

5.4 Höherwertige Funktionen als verallgemeinerte Rekursion

In der Praxis wird in Haskell Rekursion oft nicht explizit ausgeschrieben, sondern durch höherwertige Funktionen abstrahiert. Funktionen wie `map`, `filter` und `foldr` kapseln wiederkehrende Rekursionsmuster und machen den Code kürzer und ausdrucksstärker. `foldr` etwa ist eine Verallgemeinerung der strukturellen Rekursion über Listen: Es nimmt eine Funktion und einen Startwert entgegen und kombiniert alle Listenelemente durch rekursive Anwendung dieser Funktion. Viele bekannte Listenoperationen – Summe, Produkt, Länge, Umkehrung – lassen sich als Spezialfall von `foldr` darstellen. Diese höherwertigen Funktionen sind selbst rekursiv implementiert; sie machen die Rekursion für den Programmierer unsichtbar, ohne das zugrunde liegende Prinzip zu ändern [3].

6 Fazit

Rekursion ist weit mehr als ein Programmiertrick – sie ist ein grundlegendes Denkprinzip, das Mathematik und Informatik seit Jahrtausenden durchzieht. Die Idee, ein Problem durch Rückführung auf eine einfachere Version desselben Problems zu lösen, begegnet uns von Euklids Algorithmus bis hin zu modernen Typsystemen.

In der funktionalen Programmierung nimmt Rekursion eine besonders zentrale Stellung ein. Da unveränderliche Werte klassische Schleifen ausschließen, ist Rekursion das einzige Wiederholungsprinzip. Gleichzeitig ermöglicht die enge Verbindung zwischen induktiven Datentypen und strukturell rekursiven Funktionen einen klaren, mathematisch fundierten Programmierstil: Die Korrektheit eines Programms lässt sich oft direkt aus der Struktur des Datentyps ablesen, und die Implementierung ergibt sich fast zwingend aus der Problemdefinition.

Konzepte wie Endrekursion und bedarfsgesteuerte Auswertung zeigen, dass Rekursion nicht nur elegant, sondern auch effizient sein kann. Haskell demonstriert eindrucksvoll, wie ein vollständig auf Rekursion und unveränderlichen Werten aufgebautes Sprachdesign zu ausdrucksstarker, wartbarer und theoretisch fundierter Software führt. Das Verständnis von Rekursion ist daher nicht nur akademisch wertvoll, sondern eine unverzichtbare Grundlage für das Denken in modernen Programmierkonzepten.

Literatur

[1] G. Lorenz, *Funktionale Modellierung und Rekursion: Praxis – Didaktik – Theorie*. München: Oldenbourg Verlag, 2012.

[2] P. Pepper, *Funktionale Programmierung in OPAL, ML, Haskell und Gofer*. Springer Verlag, 2006.

[3] M. Block und A. Neumann, *Haskell Intensivkurs*. Springer Verlag, 2011.

[4] V. Krypczyk und O. Bochkor, *Handbuch für Softwareentwickler*, 2. Aufl. Bonn: Rheinwerk Verlag, 2021.